Growing a Competency Assignment

Skill Selection:

Chosen Skill: CI/CD Pipeline with GitHub Actions & Docker

Relevance: I chose to learn CI/CD with GitHub Actions and Docker because it directly supports my capstone project and future career. Our Banner Health capstone turns photographed EKG printouts into data for cardiac risk research: capture the image, de-identify it, digitize the waveform, measure T-waves, and later score risk. Teammates change the image-handling code every sprint, yet our repository had no automated checks. This skill tests every push before it reaches a shared branch, and Docker gives every teammate and server the same runtime for the image-processing services we will add. For a career in backend and health-tech software, both are baseline expectations. Before this, I had never written a workflow file or a Dockerfile.

Lifelong Learning Reflection:

Importance of Lifelong Learning: Lifelong learning is essential because my tools change faster than any course: during this work, both GitHub Actions I used released new major versions and GitHub's default Ubuntu runner changed. In healthcare software, where reliability and patient privacy are critical, staying current is not optional. I also kept a habit from this work: when a default matters, check the documentation or the source. Next, I plan to deploy this container to a free-tier cloud service, the part of the course video I skipped.

Learning Process:

Resources Used: To learn CI/CD with GitHub Actions and Docker, I utilized the following resources:

1. **Online Course:** "CI/CD Pipeline with GitHub Actions & Docker" by Tanuj Malode (YouTube, 40 min) - It showed the full pipeline: push, test and build in a workflow, package as a Docker image, deploy to AWS.
2. **Tutorials:** FastAPI's "Request Files" tutorial and the actions/checkout and actions/setup-python READMEs - These covered file uploads, action versions, and caching.
3. **Documentation:** GitHub Docs "Workflow syntax" and "Using jobs" pages - These defined triggers, permissions, and how needs gates one job on another.
4. **Other Resources:** Starlette and Pillow source code - I read these to learn why large uploads went to disk and how image size limits work.

Learning Steps:

1. **Step 1:** Watched the course video and read GitHub's workflow and jobs documentation.
2. **Step 2:** Built a FastAPI upload service with 14 pytest tests that use synthetic EKG-style images.
3. **Step 3:** Installed Docker Desktop, wrote a Dockerfile (slim base, pinned dependencies, non-root user, health check), and ran the container.
4. **Step 4:** Wrote a workflow that tests every push and pull request and builds the image only if the tests pass, then broke a test to confirm the build is blocked (6.5 hours total).

Demonstration of Skill:

Evidence of Learning: I created an EKG image upload API for our capstone with an automated test-and-build pipeline. POST /uploads returns an upload ID, format, dimensions, and a resolution check using our team's existing rule (long side of at least 2,376 pixels). It reads the format from the file's bytes, rejects empty, non-PNG/JPEG, damaged, and oversized files, and never stores or logs images. Below are screenshots of the finished product:

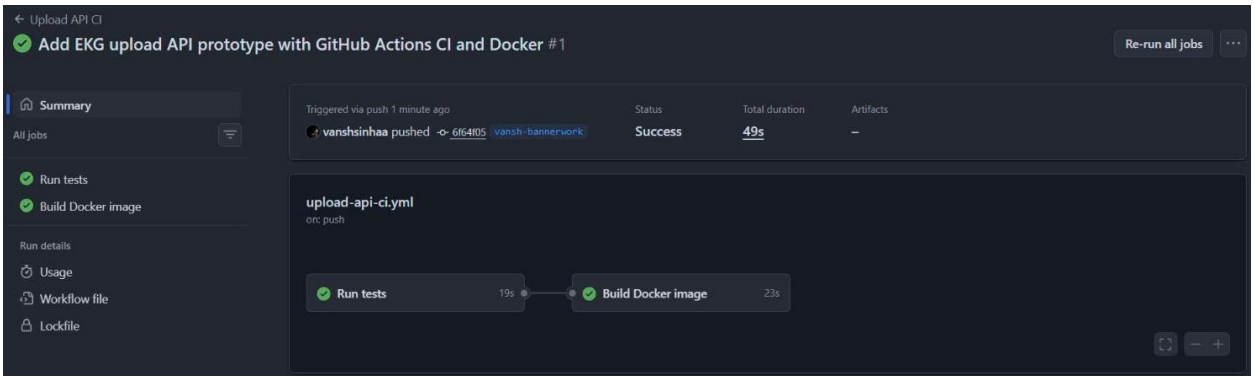


Figure 1: On every push, GitHub Actions runs the tests, then builds the Docker image and checks its /health endpoint. Both jobs passed in 49 seconds.

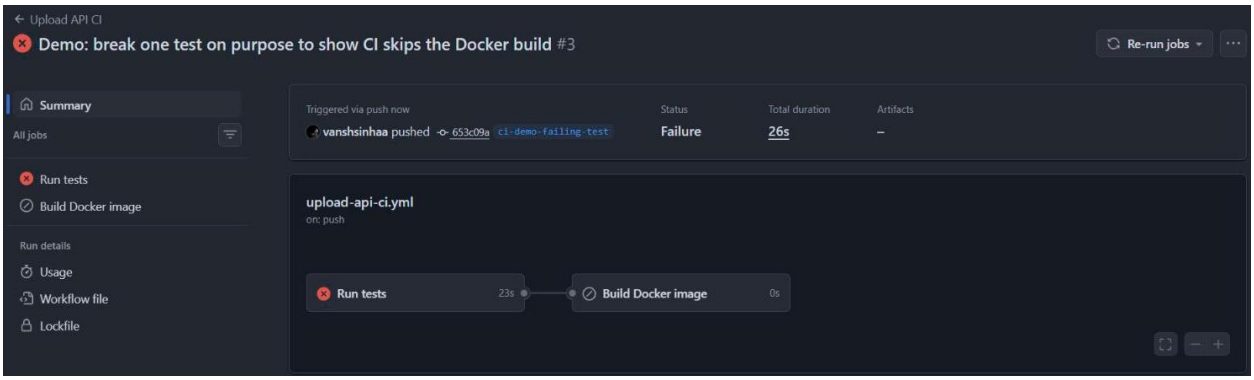


Figure 2: With one test broken on purpose, the test job fails and GitHub skips the Docker build, so failing code is never packaged.

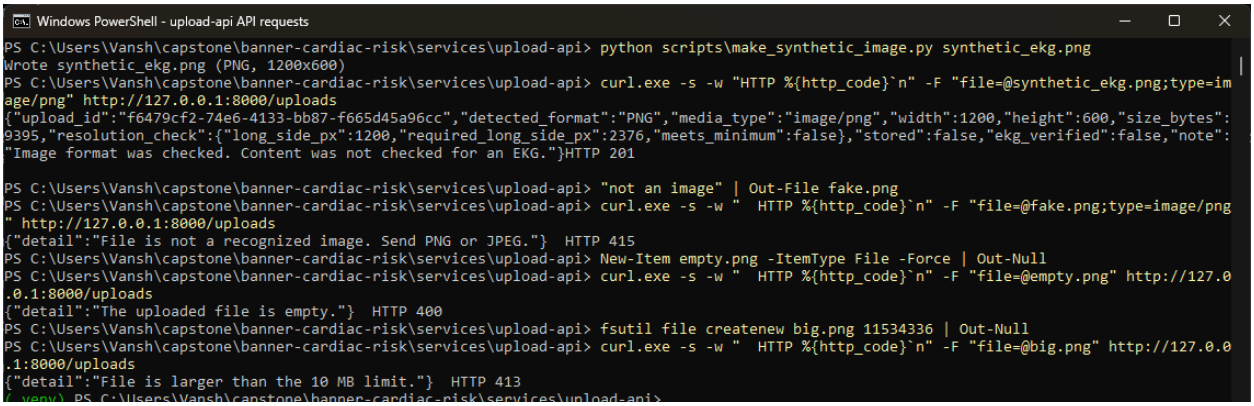


Figure 3: The containerized API accepts a synthetic EKG-style PNG (201) and rejects a fake image (415), an empty file (400), and an 11 MB file (413).

Alternatively, you can access the project via this link: https://github.com/vanshsinhaa/banner-cardiac-risk/tree/vansh-bannerwork/services/upload-api

Significance: This project is significant because it is the first server-side piece of our own capstone pipeline, not a tutorial app. It reuses our team's resolution rule so the API and capture screen agree, and its design follows our clinical context: images stay in memory, nothing is logged, test images are synthetic, and every response states the image was not verified as an EKG. Unlike a simple tutorial, this project required me to stop the framework from silently writing uploads over 1 MB to disk, reject truncated JPEGs that pass Pillow's quick check, and prove with a failing test that the pipeline blocks bad code. It does not de-identify, extract waveforms, or predict risk.

Reflection:

Challenges Faced: While learning CI/CD with GitHub Actions and Docker, I encountered several challenges, including problems that never showed up as errors. For example, FastAPI's framework quietly writes uploads over 1 MB to disk; I found this only by reading its source, raised the limit, and confirmed a 9 MB upload stayed in memory. Pillow's verify() also accepted a truncated JPEG, so I added a full decode and a test. These challenges taught me the importance of testing edge cases on purpose and checking defaults.

What I Enjoyed: I enjoyed watching manual steps become automatic. It was particularly satisfying to break a test on purpose and see the Docker job marked "skipped": needs became a safety net I could see. This experience has motivated me to extend this workflow to our Next.js app and, once digitizing exists, to check its accuracy in CI.